TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

C# PROGRAMMING LANGUAGE

Bài tập lớn:

InteractHub

Môn học:	C# and .NET Development
Học kỳ:	Spring 2026
Thời hạn:	April 19, 2026

TP. HỒ CHÍ MINH, NĂM 2026

Assignment Overview

Total Points: 10 points
Type: Individual Project
Technology Stack: TypeScript/JavaScript (Frontend) + ASP.NET Core (Backend)
Duration: [7 weeks recommended]

Contents

1 Introduction **4**
1.1 Learning Objectives 4
1.2 Application Description 4

2 Technical Requirements **5**
2.1 Technology Stack 5
2.1.1 Frontend 5
2.1.2 Backend 5
2.1.3 Cloud & DevOps 5

3 Assignment Requirements (10 Points) **6**
3.1 Frontend Requirements (4 Points) 6
3.1.1 Requirement F1: React Component Architecture & Responsive Design (1 Point) 6
3.1.2 Requirement F2: State Management & API Integration (1 Point) 7
3.1.3 Requirement F3: React Forms & Validation (1 Point) 7
3.1.4 Requirement F4: Routing, Protected Routes & Dynamic Features (1 Point) 8
3.2 Backend Requirements (4 Points) 10
3.2.1 Requirement B1: Database Design and Entity Framework (1 Point) 10
3.2.2 Requirement B2: RESTful API Controllers & DTOs (1 Point) . . 11
3.2.3 Requirement B3: JWT Authentication & Authorization (1 Point) 12
3.2.4 Requirement B4: Business Logic and Services Layer (1 Point) . . 13
3.3 Testing Requirements (1 Point) 14
3.3.1 Requirement T1: Unit Testing (1 Point) 14
3.4 CI/CD and Cloud Deployment (1 Point) 15
3.4.1 Requirement D1: Azure Deployment and CI/CD Pipeline (1 Point) 15

4 Submission Guidelines **16**
4.1 What to Submit 16
4.2 Submission Format 16
4.3 Late Submission Policy 17

5 Grading Rubric Summary **18**

6 Resources and References **18**
6.1 Official Documentation 18
6.2 Recommended Tools 18

7 Academic Integrity **19**
7.1 Not Allowed 19
7.2 Proper Attribution 19

1 Introduction

This assignment requires you to develop a fully functional social media web application called **InteractHub**. You will implement both frontend and backend components, demonstrating your understanding of modern web development practices, database design, testing methodologies, and cloud deployment.

1.1 Learning Objectives

By completing this assignment, you will:

- Design and implement responsive user interfaces using TypeScript/JavaScript and modern CSS frameworks
- Build RESTful APIs using ASP.NET Core MVC architecture
- Work with Entity Framework Core for database operations
- Implement authentication and authorization mechanisms
- Write unit tests for critical application components
- Deploy applications to cloud infrastructure (Microsoft Azure)
- Set up CI/CD pipelines for automated deployment

1.2 Application Description

InteractHub is a social media platform that allows users to:

- Create accounts and authenticate securely
- Post status updates with text and images
- Share stories (temporary content)
- Like, comment, and share posts
- Send and manage friend requests
- Receive real-time notifications
- Manage user profiles and settings
- Track trending hashtags
- Report inappropriate content (admin moderation)

2 Technical Requirements

2.1 Technology Stack

2.1.1 Frontend

- **Framework:** React 18+ with TypeScript
- **Language:** TypeScript (strict mode enabled)
- **CSS Framework:** Tailwind CSS
- **State Management:** React Context API or Redux Toolkit
- **Routing:** React Router v6+
- **HTTP Client:** Axios or Fetch API
- **Build Tool:** Vite or Create React App
- **Additional Libraries:** React Hook Form (for forms), React Query (optional for data fetching)

Important Note: The frontend must be a Single Page Application (SPA) built with React and TypeScript. It should communicate with the backend through RESTful API endpoints using JSON format.

2.1.2 Backend

- **Framework:** ASP.NET Core 8.0 or higher Web API
- **Architecture:** RESTful API with Repository and Service patterns
- **ORM:** Entity Framework Core 8.0+
- **Database:** SQL Server
- **Authentication:** JWT (JSON Web Tokens) with ASP.NET Core Identity
- **Authorization:** Role-based and Policy-based authorization
- **API Documentation:** Swagger/OpenAPI
- **CORS:** Configured for React frontend
- **Real-time:** SignalR (for notifications)

2.1.3 Cloud & DevOps

- **Cloud Platform:** Microsoft Azure
- **CI/CD:** Azure DevOps or GitHub Actions
- **Storage:** Azure Blob Storage (for images)

3 Assignment Requirements (10 Points)

3.1 Frontend Requirements (4 Points)

3.1.1 Requirement F1: React Component Architecture & Responsive Design (1 Point)

Description: Build a component-based React application with TypeScript, following best practices for component structure, props typing, and responsive design.

Specific Tasks:

- Create reusable React components with proper TypeScript interfaces
- Implement functional components with React Hooks (useState, useEffect, useContext, etc.)
- Use Tailwind CSS for responsive, mobile-first design
- Organize components in a logical folder structure (components, pages, layouts, utils)
- Implement custom hooks for reusable logic
- Create a responsive navigation system that adapts to screen sizes
- Ensure all components are mobile-friendly

Deliverables:

- At least 15 React components with TypeScript interfaces
- Component hierarchy documentation (tree structure)
- Responsive navigation bar using React Router
- Custom hooks for shared functionality
- Screenshots showing responsive design on different devices

Evaluation Criteria:

- Component architecture and reusability (35%)
- TypeScript typing and interfaces (25%)
- React Hooks usage and best practices (25%)
- Responsive design implementation (15%)

3.1.2 Requirement F2: State Management & API Integration (1 Point)

Description: Implement proper state management using React Context API or Redux, and integrate with backend RESTful APIs using Axios.

Specific Tasks:

- Set up React Context API or Redux Toolkit for global state management
- Create API service layer with Axios for HTTP requests
- Implement authentication state management (login, logout, token storage)
- Manage application state (posts, users, notifications, friends)
- Handle API loading states, errors, and success responses
- Implement JWT token storage and automatic header injection
- Create API interceptors for authentication and error handling
- Implement optimistic UI updates for better UX

Deliverables:

- Context providers or Redux store configuration
- API service files with typed responses
- Authentication context/slice with login/logout actions
- Custom hooks for API calls (e.g., usePosts, useAuth)
- Loading and error state handling across components
- TypeScript interfaces for all API responses

Evaluation Criteria:

- State management implementation (30%)
- API integration and error handling (30%)
- TypeScript typing for API responses (20%)
- Authentication flow (20%)

3.1.3 Requirement F3: React Forms & Validation (1 Point)

Description: Implement forms using React Hook Form with comprehensive validation, TypeScript typing, and excellent user experience.

Specific Tasks:

- Use React Hook Form for all forms (registration, login, post creation, profile update)
- Implement client-side validation with clear error messages
- Add custom validation rules (password strength, email format, file types)

- Create reusable form input components (TextInput, FileInput, etc.)
- Implement real-time validation feedback
- Add file upload with preview functionality
- Show loading states during form submission
- Display success/error messages after API responses

Deliverables:

- Registration form with validation (username, email, password)
- Login form with error handling
- Post creation form with image upload and preview
- Profile update form
- Reusable form components with TypeScript props
- Password strength indicator
- Form validation schemas/rules

Evaluation Criteria:

- React Hook Form implementation (30%)
- Validation completeness and accuracy (30%)
- User experience and error messages (25%)
- Component reusability (15%)

3.1.4 Requirement F4: Routing, Protected Routes & Dynamic Features (1 Point)

Description: Implement React Router with protected routes, dynamic content loading, search functionality, and performance optimizations.

Specific Tasks:

- Set up React Router v6 with nested routes
- Implement protected routes requiring authentication
- Create route guards that redirect unauthenticated users to login
- Implement search functionality with debouncing
- Add pagination or infinite scroll for post feed
- Implement lazy loading for routes and images
- Create loading skeletons for better perceived performance

- Add real-time notifications using SignalR client
- Implement client-side caching for frequently accessed data

Deliverables:

- React Router configuration with protected routes
- Authentication guard component/hook
- Search component with debounced API calls
- Pagination or infinite scroll component
- Lazy-loaded route components
- Loading skeletons for posts, users, etc.
- SignalR integration for real-time notifications

Evaluation Criteria:

- Routing implementation and protection (30%)
- Search and filtering functionality (25%)
- Performance optimizations (25%)
- Real-time features integration (20%)

3.2 Backend Requirements (4 Points)

3.2.1 Requirement B1: Database Design and Entity Framework (1 Point)

Description: Design and implement a normalized database schema using Entity Framework Core with proper relationships and constraints.

Specific Tasks:

- Design database schema with at least 8 related entities
- Implement DbContext with proper configurations
- Create Entity Framework migrations
- Define relationships (One-to-Many, Many-to-Many)
- Implement data annotations and Fluent API configurations
- Seed initial data for testing

Required Entities:

- User (AspNetUsers with Identity)
- Post
- Comment
- Like
- Friendship
- Story
- Notification
- Hashtag
- PostReport

Deliverables:

- Database diagram showing entity relationships
- Entity class files with proper annotations
- DbContext implementation
- At least 3 migration files
- Seed data configuration

Evaluation Criteria:

- Database normalization (30%)
- Proper relationship definitions (30%)
- Migration implementation (20%)
- Data validation constraints (20%)

3.2.2 Requirement B2: RESTful API Controllers & DTOs (1 Point)

Description: Implement RESTful Web API controllers that return JSON responses, with proper HTTP methods, DTOs (Data Transfer Objects), and comprehensive API documentation.

Specific Tasks:

- Create API controllers with [ApiController] attribute (AuthController, PostsController, UsersController, FriendsController, StoriesController, NotificationsController)
- Implement CRUD operations returning JSON responses (not views)
- Use proper HTTP verbs and status codes (200, 201, 400, 401, 404, 500)
- Create DTOs/ViewModels for request and response data
- Implement model validation with DataAnnotations
- Configure CORS to allow requests from React frontend
- Add Swagger/OpenAPI documentation
- Implement standardized API response format (success, data, errors)

Deliverables:

- At least 6 API controllers with [ApiController] and [Route] attributes
- At least 20 API endpoints total
- Request DTOs and Response DTOs for each endpoint
- CORS configuration in Program.cs
- Swagger UI accessible at /swagger endpoint
- Consistent API response structure
- API documentation with example requests/responses

Evaluation Criteria:

- RESTful API design and HTTP status codes (30%)
- DTO implementation and validation (25%)
- CORS and API configuration (20%)
- Swagger documentation quality (15%)
- Response format consistency (10%)

3.2.3 Requirement B3: JWT Authentication & Authorization (1 Point)

Description: Implement JWT (JSON Web Token) authentication with ASP.NET Core Identity for secure API access from React frontend.

Specific Tasks:

- Configure ASP.NET Core Identity with custom User entity
- Implement JWT token generation on successful login
- Create API endpoints: POST /api/auth/register, POST /api/auth/login
- Configure JWT authentication middleware with bearer token validation
- Implement role-based authorization (User, Admin roles)
- Protect API endpoints with [Authorize] attribute
- Return JWT token in login response for client-side storage
- Implement token refresh mechanism (optional but recommended)
- Add claims-based authorization for user-specific data

Deliverables:

- AuthController with Register/Login endpoints returning JWT
- JWT configuration in Program.cs (secret key, issuer, audience, expiration)
- User entity extending IdentityUser with additional properties
- Role seeding (User, Admin) in database

Authorize attributes on protected endpoints

- JWT token generation service/helper
- Token validation and claims extraction

Evaluation Criteria:

- JWT generation and validation (35%)
- Authentication endpoints implementation (30%)
- Role-based authorization (20%)
- Security configuration (15%)

3.2.4 Requirement B4: Business Logic and Services Layer (1 Point)

Description: Implement a service layer to separate business logic from controllers, following SOLID principles.

Specific Tasks:

- Create service interfaces and implementations
- Implement at least 5 service classes (PostsService, FriendsService, etc.)
- Use dependency injection for service registration
- Implement repository pattern for data access
- Add business logic for complex operations (friend requests, notifications)
- Implement file upload service for Azure Blob Storage
- Create helper classes and extensions

Deliverables:

- Service interface definitions
- Service class implementations
- Dependency injection configuration in Program.cs
- Business logic for key features
- File upload/storage service
- Unit-testable code structure

Evaluation Criteria:

- Separation of concerns (30%)
- SOLID principles adherence (30%)
- Dependency injection usage (20%)
- Code reusability and maintainability (20%)

3.3 Testing Requirements (1 Point)

3.3.1 Requirement T1: Unit Testing (1 Point)

Description: Write unit tests for critical backend services and frontend components to ensure code reliability.

Specific Tasks:

- Create a test project using xUnit or NUnit
- Write unit tests for at least 3 service classes
- Test authentication and authorization logic
- Mock dependencies using Moq or similar framework
- Achieve at least 60% code coverage for services
- Test edge cases and error scenarios
- Write integration tests for critical workflows

Deliverables:

- Test project with proper structure
- At least 15 unit test methods
- Test coverage report
- Tests for positive and negative scenarios
- Mock configurations and test data
- Testing documentation

Evaluation Criteria:

- Test coverage and completeness (35%)
- Test case quality and scenarios (30%)
- Proper use of mocking frameworks (20%)
- Test documentation (15%)

3.4 CI/CD and Cloud Deployment (1 Point)

3.4.1 Requirement D1: Azure Deployment and CI/CD Pipeline (1 Point)

Description: Deploy the application to Microsoft Azure with automated CI/CD pipeline for continuous integration and deployment.

Specific Tasks:

- Create Azure account and resource group
- Deploy application to Azure App Service
- Configure Azure SQL Database
- Set up Azure Blob Storage for file uploads
- Create CI/CD pipeline using Azure DevOps or GitHub Actions
- Configure environment variables and connection strings
- Implement automated build and deployment on git push
- Set up application monitoring and logging

Deliverables:

- Live application URL on Azure App Service
- CI/CD pipeline configuration files (YAML)
- Azure resource configuration documentation
- Connection strings and environment setup guide
- Deployment logs showing successful builds
- Application Insights or monitoring setup
- Deployment documentation with screenshots

Evaluation Criteria:

- Successful Azure deployment (30%)
- CI/CD pipeline implementation (30%)
- Environment configuration (20%)
- Documentation quality (20%)

4 Submission Guidelines

4.1 What to Submit

1. **Source Code:**
   - Complete Visual Studio solution (.sln file)
   - All project files and dependencies
   - Git repository URL (GitHub/Azure Repos)
   - .gitignore file (exclude bin, obj, packages)
2. **Database:**
   - SQL script for database creation
   - Entity Framework migration files
   - Seed data script
3. **Documentation:**
   - README.md with project overview
   - Setup and installation instructions
   - Database diagram
   - API documentation or endpoints list
   - Screenshots of key features (at least 10)
   - Video demonstration (5-10 minutes, optional but recommended)
4. **Testing:**
   - Test project with all test cases
   - Test coverage report
   - Test execution results
5. **Deployment:**
   - Live application URL
   - CI/CD pipeline configuration
   - Deployment documentation
   - Azure resource list and configuration

4.2 Submission Format

- **Code Repository:** Share GitHub/Azure DevOps repository link with instructor access
- **Documentation:** Submit PDF document or well-formatted README.md
- **Compressed Archive:** Zip file containing all materials (max 50MB, exclude node_modules, bin, obj)
- **Submission Portal:** Upload to designated learning management system

4.3 Late Submission Policy

- On-time submission: 100% of earned points
- 1-3 days late: 10% penalty
- 4-7 days late: 20% penalty
- More than 7 days late: 50% penalty
- No submissions accepted after 14 days without prior approval

5 Grading Rubric Summary

Category	Points	Weight
Frontend - Responsive UI	1	10%
Frontend - Interactive Components	1	10%
Frontend - Form Validation	1	10%
Frontend - Dynamic Content	1	10%
Frontend Subtotal	**4**	**40%**
Backend - Database & EF	1	10%
Backend - RESTful API	1	10%
Backend - Authentication	1	10%
Backend - Business Logic	1	10%
Backend Subtotal	**4**	**40%**
Testing	1	10%
CI/CD & Cloud Deployment	1	10%
Total	**10**	**100%**

Table 1: Assignment Grading Distribution

6 Resources and References

6.1 Official Documentation

- ASP.NET Core Documentation: `https://docs.microsoft.com/aspnet/core`
- Entity Framework Core: `https://docs.microsoft.com/ef/core`
- TypeScript Documentation: `https://www.typescriptlang.org/docs`
- Tailwind CSS: `https://tailwindcss.com/docs`
- Azure App Service: `https://docs.microsoft.com/azure/app-service`

6.2 Recommended Tools

- Visual Studio 2022 or VS Code
- SQL Server Management Studio (SSMS)
- Postman (API testing)
- Azure Data Studio
- Git for version control

7 Academic Integrity

7.1 Not Allowed

- Copying code from other students
- Submitting purchased or downloaded complete projects
- Using AI to generate entire features without understanding
- Sharing your code with other students

7.2 Proper Attribution

- Comment your code to explain complex logic
- Cite external libraries and frameworks used
- Document any code snippets adapted from online sources
- Explain any third-party integrations

Note: Plagiarism will result in a zero grade for the assignment and may lead to further academic consequences.

THE END